An inconvenience with PackBits is that the nine bits must be reformatted into the standard eight bit bytes used in computer storage and transmission. A useful modification to this scheme can be made when the input is restricted to be ASCII text. As shown in Table 27-2, each ASCII character is usually stored as a full byte (eight bits), but really only uses *seven* of the bits to identify the character. In other words, the values 127 through 255 are not defined with any standardized meaning, and do not need to be stored or transmitted. This allows the eighth bit to indicate if run-length encoding is in progress.

Huffman Encoding

This method is named after D.A. Huffman, who developed the procedure in the 1950s. Figure 27-2 shows a histogram of the byte values from a large ASCII file. More than 96% of this file consists of only 31 characters: the lower case letters, the space, the comma, the period, and the carriage return. This observation can be used to make an appropriate compression scheme for this file. To start, we will assign each of these 31 common characters a five bit binary code: 00000 = "a", 00001 = "b", 00010 = "c", etc. This allows 96% of the file to be reduced in size by 5/8. The last of the five bit codes, 11111, will be a flag indicating that the character being transmitted is not one of the 31 common characters. The next eight bits in the file indicate what the character is, according to the standard ASCII assignment. This results in 4% of the characters in the input file requiring 5+8=13 bits. The idea is to assign frequently used characters fewer bits, and seldom used characters

TABLE 27-2
ASCII codes. This is a long established standard for allowing letters and numbers to be represented in digital form. Each printable character is assigned a number between 32 and 127, while the numbers between 0 and 31 are used for various control actions. Even though only 128 codes are defined, ASCII characters are usually stored as a full byte (8 bits). The undefined values (128 to 255) are often used for Greek letters, math symbols, and various geometric patterns; however, this is not standardized. Many of the control characters (0 to 31) are based on older communications networks, and are not applicable to computer technology.

0	null	32	space	64	@	96	`
1	start heading	33	!	65	A	97	a
2	start of text	34	"	66	B	98	b
3	end of text	35	#	67	C	99	c
4	end of xmit	36	$	68	D	100	d
5	enquiry	37	%	69	E	101	e
6	acknowledge	38	&	70	F	102	f
7	bell, beep	39	'	71	G	103	g
8	backspace	40	(	72	H	104	h
9	horz. tab	41	)	73	I	105	i
10	line feed	42	*	74	J	106	j
11	vert. tab, home	43	+	75	K	107	k
12	form feed, cls	44	,	76	L	108	l
13	carriage return	45	-	77	M	109	m
14	shift out	46	.	78	N	110	n
15	shift in	47	/	79	O	111	o
16	data line esc	48	0	80	P	112	p
17	device control 1	49	1	81	Q	113	q
18	device control 2	50	2	82	R	114	r
19	device control 3	51	3	83	S	115	s
20	device control 4	52	4	84	T	116	t
21	negative ack.	53	5	85	U	117	r
22	synch. idle	54	6	86	V	118	v
23	end xmit block	55	7	87	W	119	w
24	cancel	56	8	88	X	120	x
25	end of medium	57	9	89	Y	121	y
26	substitute	58	:	90	Z	122	z
27	escape	59	;	91	[	123	{
28	file separator	60	<	92	\	124	|
29	group separator	61	=	93	]	125	}
30	record separator	62	>	94	^	126	~
31	unit separator	63	?	95	_	127	del